- Employees need to be organized into shifts.
- Customers may browse and order from a website.
- Central warehouse supplies shop inventory.
- Central warehouse also fulfills online orders.

From these requirements, we can identify some entities (real things that need to be represented somehow in our system) …

- Book
- Shop
- Customer
- Employee
- Manager
- Warehouse
- Inventory

… and some actions (with their related entities) …

- Schedule shifts (employee, manager, shop)
- Order new inventory (manager, shop, warehouse)
- Make shop sales (book, inventory, shop, potentially customer too)
- Make online sales (book, customer, inventory, warehouse), etc.

This is an example of *modular decomposition*. We have taken the problem domain information and created a set of objects and interactions to represent it.

Without even considering the constraints on the solution space, we're already looking at hundreds (perhaps thousands) of lines of code across many classes. We could dive in and begin creating code at this point, but just using code and written requirements to manage our design would be quite a challenge.

It would be far better at this point to spend some time considering the classes we need, how they're structured, and how they interact. Finding problems at the design stage is less of a headache than during implementation. Wading through source code to build an overview of how our system functions is a difficult proposition. Visually designing classes (and their interactions) makes much more sense.